

동아줄

AI 전략 리스크 진단 서비스 — 전체 코드 해설집

DBR·HBR·HBS 15,000건+ 사례 데이터베이스 기반의 전략 리스크 진단 서비스입니다.
본 문서는 발표 전 전체 코드 흐름을 이해하기 위한 레이어별 핵심 코드 해설집입니다.

레이어	기술 스택
Frontend	React 18 + TypeScript + Tailwind CSS + D3.js
Backend	Node.js / Express + JWT Auth (Access 15분 / Refresh 7일)
AI Server	FastAPI (Python) — SBERT + FAISS + MLP + GPT-4o-mini
ML Pipeline	Kiwi 형태소 분석 → 라벨링 → SBERT 임베딩 → FAISS → MLP → UMAP
DB	MySQL + FAISS Index (15,451건 기사)
Auth	Kakao / Naver / Google OAuth 2.0

목 차

- 1장 시스템 아키텍처 전체 흐름
- 2장 NLP 파이프라인 — 데이터처리/
 - 2.1 ① 전처리 (preprocess.py)
 - 2.2 ② 라벨링 (label.py)
 - 2.3 ③ 리스크 모델 (risk_model.py)
 - 2.4 ④ UMAP + K-means (umap_cluster.py)
- 3장 AI 서버 — ai_server/
 - 3.1 FastAPI 앱 + 서버 시작 (main.py)
 - 3.2 /diagnose 엔드포인트 — 3-factor 스코어링
 - 3.3 /report 엔드포인트 — GPT RAG 리포트
 - 3.4 Pydantic 스키마 (schemas.py)
 - 3.5 전략 프레임워크 KB (frameworks.py)
 - 3.6 트렌드 컨텍스트 (trend_context.py)
- 4장 백엔드 — backend/ (Node.js / Express)
 - 4.1 DB 연결 (db.js)
 - 4.2 JWT 인증 + OAuth (authController.js)
 - 4.3 진단 처리 (diagnoseController.js)
 - 4.4 기사 통계 (articleController.js)
 - 4.5 기사 비교 GPT (compareController.js)
 - 4.6 API 라우트 정의
- 5장 프론트엔드 — frontend/ (React + TypeScript)
 - 5.1 API 유틸리티 (api.ts) — JWT 자동 갱신
 - 5.2 App.tsx — SPA 라우팅 + History API
 - 5.3 주요 컴포넌트 구조
- 6장 DB 스키마 핵심 테이블
- 7장 배포 구성 (NCP + Docker)

1장. 시스템 아키텍처 전체 흐름

동아줄은 3개 서버(Frontend · Backend · AI Server)가 분리된 마이크로서비스 구조입니다.

★ 핵심 흐름 사용자 전략 입력 → Backend (Node.js) → AI Server (FastAPI) → SBERT 임베딩 → FAISS 유사 사례 검색 → MLP
리스크 스코어 → GPT 리포트 → DB 저장 → Frontend 렌더링

브라우저 (React)	→ POST /api/diagnose	Backend :3001
Backend :3001	→ POST /report	AI Server :8000
AI Server :8000	→ SBERT 임베딩	jhgan/ko-sroberta-multitask (768차원)
AI Server :8000	→ FAISS 검색	faiss_with_hbs.index (15,451건, IndexFlatIP)
AI Server :8000	→ MLP predict_proba	risk_model.pkl (ROC-AUC 0.9771)
AI Server :8000	→ GPT-4o-mini	reporter.py (Few-shot + RAG, temp=0.5)
Backend :3001	→ DB 저장	MySQL (diagnosis_requests + analysis_results)
Backend :3001	→ JSON 응답	Frontend DiagnosisResult 컴포넌트

Frontend↔Backend 통신: nginx 리버스 프록시 (배포) / CORS (로컬). Frontend → /api/* → backend:3001 → /report → ai_server:8000

2장. NLP 파이프라인 — 데이터처리/

5단계 파이프라인으로 원시 CSV에서 FAISS 인덱스 + 리스크 모델 + UMAP 좌표까지 구축합니다.

단계	내용
① 전처리	Kiwi 형태소 분석 + 불용어 제거 → clean_text / tokens
② 라벨링	키워드 히트(Stage1) + SBERT 센트로이드(Stage2) → success/failure/neutral
③ 임베딩	paraphrase-multilingual-MiniLM-L12-v2 384차원 → embeddings.npy → FAISS
④ 리스크 모델	jhgan/ko-sroberta-multitask 768차원 → MLP → risk_model.pkl
⑤ UMAP+클러스터	UMAP 2D + K-means 12개 클러스터 → umap_coords_v3.parquet

2.1 ① 전처리 — preprocess.py

입력: DBR_articles.csv (11,273건), HBR_articles.csv (2,062건)

출력: DBR_preprocessed.parquet, HBR_preprocessed.parquet

추가 컬럼: clean_text, tokens, token_str, n_tokens

★ 핵심 기술 — Kiwi 형태소 분석 NNG(일반명사)/NNP(고유명사)/VV(동사)/VA(형용사)/SL(영어)/XR(어근) 6개 품사만 보존.
불용어 220개, 최소 토큰 길이 2자, 최소 문서 토큰 수 10개.

① 텍스트 전처리 파이프라인 (동아줄 - AI 전략 리스크 진단 서비스)

입력 : 크롤링/DBR_articles.csv, 크롤링/HBR_articles.csv (7컬럼)

출력 : NLP/output/{DBR,HBR}_preprocessed.parquet

처리 단계

- 1) CSV 로드 (utf-8-sig, BOM 처리)
- 2) 텍스트 정제 (URL/이메일/HTML/캡션/특수문자/공백 정규화)
- 3) Kiwi 형태소 분석 → 의미 품사만 추출 (명사/동사/형용사/외국어/어근)
- 4) 불용어 제거 + 길이 필터
- 5) 토큰 수 부족 문서 제거
- 6) parquet 저장 (원본 + clean_text + tokens + token_str + n_tokens)

실행: python NLP/preprocess.py

```
from __future__ import annotations

import re
import sys
import time
from pathlib import Path

import pandas as pd
from kiwipiepy import Kiwi

# Windows 콘솔(cp949)에서도 한글 깨지지 않게
sys.stdout.reconfigure(encoding="utf-8")

ROOT = Path(__file__).resolve().parent.parent
CRAWL_DIR = ROOT / "크롤링"
```

```

OUT_DIR = ROOT / "데이터처리" / "output"
STOPWORD_FILE = ROOT / "데이터처리" / "stopwords_ko.txt"

# Kiwi에서 유지할 품사 태그
# NNG 일반명사 / NNP 고유명사 / VV 동사 / VA 형용사 / SL 외국어(영문) / XR 어근
KEEP_TAGS = {"NNG", "NNP", "VV", "VA", "SL", "XR"}
LEMMA_TAGS = {"VV", "VA"} # 어간에 '다'를 붙여 사전형 복원
MIN_TOKEN_LEN = 2 # 토큰 최소 길이(한/영 공통)
MIN_DOC_TOKENS = 10 # 이보다 토큰이 적은 문서는 제외

# ----- 정규식 -----

RE_URL = re.compile(r"https?://WS+|wwwW.WS+")
RE_EMAIL = re.compile(r"[Ww.+~]+@[Ww-]+W.[Ww-]+")
RE_HTML_TAG = re.compile(r"<[^>]+>")
RE_HTML_ENTITY = re.compile(r"&[a-zA-Z#0-9]+;")
# [사진], (사진=...), <그림 1> 등 캡션/출처 표기 제거
RE_CAPTION = re.compile(r"[W(W(<[^W]W>){0,40}(사진|그림|자료|출처|이미지|표)[^W]W>){0,40}[W]W>]")
RE_REPORTER = re.compile(r"[가-힣]{2,4}Ws?(기자|특파원|연구원|교수|대표|기고)")
# 한글/영문/숫자/공백만 남김 (그 외 기호 → 공백)
RE_NONTEXT = re.compile(r"^[^0-9A-Za-z가-힣Ws]*")
RE_MULTISPACE = re.compile(r"Ws+")

def load_stopwords() -> set[str]:
    words: set[str] = set()
    for line in STOPWORD_FILE.read_text(encoding="utf-8").splitlines():
        line = line.strip()
        if line and not line.startswith("#"):
            words.add(line)

```

```

KEEP_TAGS = {'NNG', 'NNP', 'VV', 'VA', 'SL', 'XR'} # 보존할 품사
LEMMA_TAGS = {'VV', 'VA'} # 원형 복원 대상

def clean_text(text: str) -> str:
    text = RE_URL.sub(' ', text) # URL 제거
    text = RE_HTML_TAG.sub(' ', text) # HTML 태그 제거
    text = RE_REPORTER.sub(' ', text) # 기자 이름 제거
    text = RE_NONTEXT.sub(' ', text) # 특수문자 제거
    return RE_MULTISPACE.sub(' ', text).strip()

# Kiwi 형태소 분석 → 품사 필터 → 불용어 제거 → token_str 저장
def tokenize_corpus(kiwi, texts, stopwords):
    results = []
    for tokens_raw in kiwi.analyze(texts, batch_size=200):
        tokens = [t.form if t.tag not in LEMMA_TAGS else t.lemma
                    for t in tokens_raw[0][0]]
        if t.tag in KEEP_TAGS
        and len(t.form) >= MIN_TOKEN_LEN
        and t.form not in stopwords]
        results.append(tokens)
    return results

```

2.2 ② 라벨링 — label.py

2단계 라벨링: Stage1 키워드 히트 → Stage2 SBERT 센트로이드 재분류

항목	설명
Stage1 — 키워드	success/failure 키워드 히트 수 기반 success_ratio 계산 $\geq 0.65 \rightarrow$ success, $\leq 0.35 \rightarrow$ failure, min 3히트 필요
Stage2 — SBERT	Stage1 모호(neutral) 문서를 SBERT 센트로이드 코사인 유사도로 재분류 sim_threshold=0.15
출력 컬럼	label (success/failure/neutral), label_stage (keyword/tfidf), confidence
클래스 분포	DBR: success 86.9% / failure 4.7% / neutral 8.3%

★ 핵심 기술 — 2단계 라벨링 단순 키워드 매칭의 한계를 SBERT 센트로이드 유사도로 보완. 실패(4.7%) 클래스 불균형은 ④ 단계에서 class_weight='balanced'로 해결.

```
# Stage1: 키워드 히트 기반
SUCCESS_KEYWORDS = ['성장', '매출', '확대', '성공', '혁신', '수익', ...]
FAILURE_KEYWORDS = ['실패', '위기', '철수', '손실', '부도', '적자', ...]

def stage1_label(text):
    s = sum(kw in text for kw in SUCCESS_KEYWORDS)
    f = sum(kw in text for kw in FAILURE_KEYWORDS)
    total = s + f
    if total < 3: return 'neutral', 0.5
    ratio = s / total
    if ratio >= 0.65: return 'success', ratio
    if ratio <= 0.35: return 'failure', 1-ratio
    return 'neutral', ratio

# Stage2: SBERT 센트로이드 재분류 (neutral → success/failure)
success_centroid = model.encode(success_docs).mean(axis=0)
failure_centroid = model.encode(failure_docs).mean(axis=0)

def stage2_label(emb):
    sim_s = cosine_similarity([emb], [success_centroid])[0][0]
    sim_f = cosine_similarity([emb], [failure_centroid])[0][0]
    if abs(sim_s - sim_f) > 0.15:
        return ('success' if sim_s > sim_f else 'failure'), max(sim_s, sim_f)
    return 'neutral', 0.5
```

2.3 ③ 리스크 모델 — risk_model.py

입력: SBERT 768차원 임베딩 / DBR+HBR+NAVER 68,800건 / neutral 제외

출력: risk_model.pkl (MLP) — risk_score = P(failure|embedding)

★ 핵심 기술 — MLP 채택 이유 모델 비교: MLP > LightGBM > XGBoost > LogisticRegression. Test ROC-AUC=0.9771 / Failure Precision=0.71 / Recall=0.87 / F1=0.78. class_weight='balanced'로 실패(4.7%) 불균형 보정.

```
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import StratifiedKFold

# MLP: hidden=(512,128), activation=relu, class_weight=balanced
```

```

mlp = MLPClassifier(
hidden_layer_sizes=(512, 128),
activation='relu',
max_iter=300,
random_state=42,
)

# 5-fold Stratified CV + F1 최대화 임계값 탐색
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
best_threshold = 0.33 # 채택된 최적 임계값

# 저장
import pickle
with open('output/risk_model.pkl', 'wb') as f:
pickle.dump({'model': mlp, 'threshold': best_threshold,
'classes': mlp.classes_.tolist(), f)

# 사용 시: risk_score = model.predict_proba(emb)[0, fail_col]

```

2.4 ④ UMAP + K-means — umap_cluster.py

입력: 768차원 SBERT 임베딩 / 출력: umap_coords_v3.parquet, cluster_info_v3.parquet

★ 핵심 기술 — 768차원 K-means 기존 2D UMAP 좌표 기반 K-means → HBR 영어 기사 언어 클러스터 분리 문제 발생. 개선: 768차원 임베딩 기반 K-means → 12개 주제 기반 의미 클러스터 (AI/디지털, 재무/투자, 마케팅 등).

```

import umap
from sklearn.cluster import KMeans

# Step 1: 768차원 K-means로 의미 클러스터 먼저 결정
kmeans = KMeans(n_clusters=12, random_state=42, n_init=20)
cluster_ids = kmeans.fit_predict(embeddings_768d)

# Step 2: UMAP으로 2D 시각화 (cluster_id는 K-means 결과 사용)
reducer = umap.UMAP(
n_components=2,
n_neighbors=15,
min_dist=0.1,
metric='cosine',
random_state=42
)

coords_2d = reducer.fit_transform(embeddings_768d) # 약 24초

# 출력: title/url/label/umap_x/umap_y/cluster_id
df['umap_x'] = coords_2d[:, 0]
df['umap_y'] = coords_2d[:, 1]
df['cluster_id'] = cluster_ids + 1 # 0-11 → 1-12 (MySQL auto_increment 대응)

```

3장. AI 서버 — ai_server/ (FastAPI + Python)

FastAPI 기반 ML 서버. 서버 시작 시 모든 모델/인덱스를 메모리에 로드합니다.

엔드포인트	설명
POST /diagnose	SBERT 임베딩 → FAISS 검색 → 3-factor 리스크 스코어
POST /report	/diagnose + GPT RAG 리포트 (Few-shot 3개)
POST /diagnose/global	HBS 전용 FAISS → 해외 사례 5건 반환 (DB 저장 없음)
GET /health	서버 상태 + 로드된 모델 정보
GET /clusters	클러스터 12개 목록 (시맨틱맵용)
GET /semantic-map	UMAP 전체 포인트 15,000건+ 반환 (D3.js 시각화용)

3.1 FastAPI 앱 + 서버 시작 (lifespan) — main.py

★ 핵심 기술 — lifespan FastAPI lifespan 컨텍스트 매니저로 서버 시작 시 1회 모델 로드. SBERT(768차원) → FAISS(HBS 포함 우선) → MLP → 이상감지 모델 → 메타데이터 → UMAP v3 → 클러스터 실패율 사전계산.

```
# ai_server/main.py — lifespan (서버 시작 시 1회 실행)
@asynccontextmanager
async def lifespan(app: FastAPI):
    global _sbert, _faiss_index, _risk_model, _meta, _umap, _cluster_risk_map

    # 1. SBERT 로드 (768차원, 한국어 특화)
    _sbert = SentenceTransformer('jhgan/ko-sroberta-multitask')
    init_frameworks(_sbert) # 전략 프레임워크 KB 임베딩

    # 2. FAISS (DBR+HBR+HBS 15,451건 우선, 없으면 13,335건 fallback)
    if (OUT_DIR / 'faiss_with_hbs.index').exists():
        index_bytes = (OUT_DIR / 'faiss_with_hbs.index').read_bytes()
        _faiss_index = faiss.deserialize_index(
            np.frombuffer(index_bytes, dtype=np.uint8)) # 한글 경로 버그 우회

    # 3. MLP 리스크 모델
    _risk_model = pickle.load(open(OUT_DIR / 'risk_model.pkl', 'rb'))

    # 4. 클러스터별 실패율 사전계산 (_cluster_risk_map)
    for cid, grp in _umap.groupby('cluster_id'):
        fail = int((grp['label_name'] == 'failure').sum())
        _cluster_risk_map[int(cid)] = round(fail / len(grp), 4)

    init_trend_context(OUT_DIR) # NAVER 트렌드 키워드 로드
    yield
```

FAISS 한글 경로 버그: faiss.write_index()의 C++ 백엔드가 한글 경로 미지원 → faiss.serialize_index() bytes → Path.write_bytes()로 우회 저장.

3.2 /diagnose — 3-factor 리스크 스코어링

★ 핵심 기술 — 3-factor 스코어링 $base = 0.35 \times model_score + 0.45 \times case_score + 0.20 \times cluster_risk$ → $risk_score = base \times reliability + 0.5 \times (1 - reliability)$ ($reliability = \min(max_sim/0.65, 1.0)$, 유사도 낮으면 0.5 방향 수축)


```
# POST /diagnose — 3-factor 리스크 스코어링
q_emb = _sbert.encode([req.text], normalize_embeddings=True).astype(np.float32)

# 실패 사례 보장: 넓은 풀(100건+)에서 검색 후 실패 최소 3건 보장
pool_k = min(max(req.top_k * 20, 100), _faiss_index.ntotal)
scores, ids = _faiss_index.search(q_emb, pool_k)

# Factor 1: MLP 모델 P(failure)
model_score = float(_risk_model['model'].predict_proba(q_emb)[0, fail_col])

# Factor 2: 유사 사례 신뢰도 가중 실패 비율
case_score = sum(e['sim'] * e['conf'] for e in failure_cases) /
sum(e['sim'] * e['conf'] for e in all_cases + [1e-9])

# Factor 3: 클러스터 실패율
cluster_risk = _cluster_risk_map.get(query_cluster_id, 0.35)

# 신뢰도 보정 (유사도 낮으면 0.5 방향으로 수축)
max_sim = max(e['sim'] for e in selected) if selected else 0.0
reliability = min(max_sim / 0.65, 1.0)

base_score = 0.35*model_score + 0.45*case_score + 0.20*cluster_risk
risk_score = base_score * reliability + 0.5 * (1 - reliability)
```

3.3 /report — GPT RAG 리포트 (reporter.py)

LangChain + GPT-4o-mini (temperature=0.5). 8개 필드 JSON 출력.

Few-shot 3개 예시: 저가전략(high) / 프리미엄전략(low) / AI SaaS(medium)

★ 핵심 기술 — Few-shot + RAG 프롬프팅 유사 사례 Top-K를 벡터 유사도 기반으로 검색해 GPT 프롬프트에 주입(RAG). 추가로 ① NAVER 트렌드 키워드 ② 전략 프레임워크 컨텍스트 주입. 처방('이렇게 해라') 대신 패턴 경보('이런 조건에서 이런 결과') 형태 강제.

```
# ai_server/reporter.py
HUMAN_PROMPT = "[분석 대상 전략] {strategy_text}
[리스크 스코어] {risk_score} / 1.0 (등급: {risk_level})
[유사 사례 Top-{k}] {similar_cases} # FAISS 검색 결과 주입 (RAG)
{trend_section} # NAVER 트렌드 컨텍스트
{web_trend_section} # 웹 검색 트렌드
{framework_section}" # 관련 경영 프레임워크

# 출력 JSON 8개 필드
# summary / strategy_analysis / market_context
# risk_factors[3] / risk_details[3] / improvement[3]
# framework_insight / verdict

llm = ChatOpenAI(model='gpt-4o-mini', temperature=0.5, timeout=30)
_chain = SYSTEM_PROMPT + FEWSHOT_3개 + HUMAN_PROMPT | llm | JsonOutputParser()
```

Few-shot 예시를 3개 삽입해 GPT가 JSON 구조와 패턴 경보 형태를 정확히 학습. framework_insight는 SBERT 코사인 유사도 0.35 이상인 프레임워크가 있을 때만 주입.

3.4 Pydantic 스키마 — schemas.py

```
# ai_server/schemas.py
class DiagnoseRequest(BaseModel):
```

```

text: str
top_k: int = 5

class SimilarArticle(BaseModel):
    rank: int
    title: str; url: str; label: str; similarity: float
    summary: str; category: str; published_date: str; source: str

class DiagnosisReport(BaseModel):
    summary: str
    strategy_analysis: Optional[str]
    market_context: Optional[str]
    risk_factors: List[str]
    risk_details: Optional[List[str]]
    improvement: List[str]
    framework_insight: Optional[str]
    verdict: str

class ReportResponse(DiagnoseResponse):
    report: DiagnosisReport
    query_umap_x: Optional[float]
    query_umap_y: Optional[float]
    query_cluster_id: Optional[int]

```

3.5 전략 프레임워크 KB — frameworks.py

12개 경영전략 프레임워크 텍스트를 SBERT로 임베딩 → 코사인 유사도 기반 자동 매칭.

★ 12개 프레임워크 Porter 5 Forces / 본원적 전략 / 파괴적 혁신 / 블루오션 / JTBD / BMC / 린스타트업 / 고슴도치+플라이휠 / Zero to One / 앤소프 매트릭스 / 가치사슬 / 플랫폼·네트워크

```

# ai_server/frameworks.py
FRAMEWORKS = [
    {'name': 'Porter 5 Forces', 'text': '진입장벽·공급자·구매자·대체재·경쟁 강도 5가지...'},
    {'name': '린스타트업', 'text': 'Build-Measure-Learn 순환, MVP 최소 기능 제품...'},
    # ... 12개
]

def init_frameworks(sbert_model):
    for fw in FRAMEWORKS:
        fw['emb'] = sbert_model.encode([fw['text']], normalize_embeddings=True)[0]

def find_relevant_frameworks(q_emb, top_k=2):
    results = []
    for fw in FRAMEWORKS:
        sim = float(np.dot(q_emb, fw['emb'])) # 코사인 유사도
        if sim >= 0.35:
            results.append((sim, fw))
    # Porter 계열 최대 1개 제한 (편향 방지)
    results.sort(reverse=True)
    return [fw['text'] for _, fw in results[:top_k]]

```

3.6 트렌드 컨텍스트 — trend_context.py

NAVER 뉴스 55,465건(2024~2025) TF-IDF 키워드를 23개 카테고리로 분류해 GPT 프롬프트에 주입.

```
# ai_server/trend_context.py
# trend_summaries.json (GPT 요약) 우선, trend_keywords.json (TF-IDF) fallback

CLUSTER_TO_CATEGORY = {
    'AI/디지털': ['IT/인터넷', 'AI/로봇'],
    '재무/투자': ['금융/투자', '경제'],
    '마케팅/브랜드': ['마케팅', '광고'],
    # ... 12개 클러스터 → 23개 NAVER 카테고리 매핑
}

def get_trend_context(cluster_name: str, strategy_text: str) -> str:
    # 클러스터명 → NAVER 카테고리 매핑
    cats = CLUSTER_TO_CATEGORY.get(cluster_name, ['전체'])
    keywords = []
    for cat in cats:
        keywords.extend(trend_data.get(cat, {}).get('keywords', [])[:15])
    return f'최근 시장 트렌드 키워드 (2024~2025)\n\n{' '.join(keywords[:20])}'
```

4장. 백엔드 — backend/ (Node.js / Express)

Node.js + Express. JWT 인증, 카카오/네이버/구글 OAuth, MySQL 연결, AI 서버 프록시 역할.

항목	내용
서버 포트	3001 (로컬) / Docker 내부 expose (배포)
인증	JWT Access Token 15분 + Refresh Token 7일 (DB 저장)
OAuth	Kakao / Naver / Google (passport.js 없이 직접 구현)
AI 서버 연동	axios POST /report → AI Server :8000 (timeout=180s)
DB	mysql2/promise pool (connectionLimit=10)

4.1 DB 연결 — db.js

```
// backend/src/config/db.js
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: process.env.MYSQL_HOST,
  port: process.env.MYSQL_PORT || 3306,
  user: process.env.MYSQL_USER,
  password: process.env.MYSQL_PASSWORD,
  database: process.env.MYSQL_DATABASE,
  waitForConnections: true,
  connectionLimit: 10,
  // charset 옵션 없음 — DB 서버가 latin1 기반이라 charset 변환 시 한글 garbling 발생
});

module.exports = pool;
```

charset: 'utf8mb4' 옵션 절대 추가 금지. DB 서버(campus.smhrd.com)가 latin1 기반이라 charset 변환 시 한글 garbling 발생. 팀 내 버그 이력 있음.

4.2 JWT 인증 + OAuth — authController.js

★ 핵심 기술 — JWT Refresh Token Access Token 15분 + Refresh Token 7일 (DB 저장). 401 응답 시 /api/auth/refresh 자동 재발급 → 원래 요청 재시도. 소셜 콜백 에러는 ?error=xxx_login_failed 로 리다이렉트.

```
// authController.js — JWT 발급
const accessToken = jwt.sign(payload, JWT_SECRET, { expiresIn: '15m' });
const refreshToken = jwt.sign(payload, REFRESH_SECRET, { expiresIn: '7d' });
await saveRefreshToken(userId, refreshToken); // DB에 저장
res.json({ success: true, token: accessToken, refresh_token: refreshToken, user });

// 카카오 OAuth 콜백 (직접 구현, passport 미사용)
const tokenRes = await axios.post('https://kauth.kakao.com/oauth/token', tokenParams);
const profileRes = await axios.get('https://kapi.kakao.com/v1/oidc/userinfo', {
  headers: { Authorization: `Bearer ${tokenRes.data.access_token}` }
});

// 가입/로그인 처리 후 JWT 발급 → ?token=XXX&refresh_token=YYY 로 리다이렉트
res.redirect(`${FRONTEND_URL}?token=${accessToken}&refresh_token=${refreshToken}`);
```

```
// /api/auth/refresh

const decoded = jwt.verify(refreshToken, REFRESH_SECRET);

const newAccess = jwt.sign(payload, JWT_SECRET, { expiresIn: '15m' });

res.json({ success: true, token: newAccess });
```

4.3 진단 처리 — diagnoseController.js

사용자 전략 텍스트 → AI Server /report 호출 → DB 저장 (트랜잭션) → 알림 생성.

```
// diagnoseController.js — 핵심 흐름

exports.diagnose = async (req, res) => {
  const { inputText, top_k = 5 } = req.body;

  // 1. AI 서버 /report 호출 (timeout=180s, retry 없음)
  const aiRes = await axios.post(`${AI_SERVER_URL}/report`,
    { text: inputText, top_k }, { timeout: 180000 });
  const aiData = aiRes.data;

  // 2. 클러스터명 DB 조회
  const [clusterRows] = await db.execute(
    `SELECT cluster_name FROM clusters WHERE cluster_id = ?`,
    [aiData.query_cluster_id]);

  // 3. DB 저장 (트랜잭션)
  await saveToDb(req.user.user_id, inputText, aiData, clusterName);

  // 4. 알림 생성 (비동기, 실패해도 진단 응답에 영향 없음)
  createNotification(userId, '진단 완료', ...).catch(() => {});

  res.json({ success: true, data: aiData });
};

// saveToDb — 트랜잭션
async function saveToDb(userId, inputText, aiData, clusterName) {
  const conn = await db.getConnection();
  await conn.beginTransaction();

  // diagnosis_requests INSERT → analysis_results INSERT → similar_article_matches INSERT
  // report_json (MEDIUMTEXT), query_umap_x/y, query_cluster_id 포함 저장
  await conn.commit();
}
```

4.4 기사 통계 API — articleController.js

```
// GET /api/articles/stats

exports.getArticleStats = async (req, res) => {
  const [[totals]] = await db.execute(
    `SELECT COUNT(*) total,
    SUM(al.label='success') success_count,
    SUM(al.label='failure') failure_count
    FROM articles a LEFT JOIN article_labels al ON a.article_id=al.article_id`);

  // yearly_trend: YEAR(published_at) GROUP BY label
  const [yearly] = await db.execute(
    `SELECT YEAR(a.published_at) yr, al.label, COUNT(*) cnt
```

```

FROM articles a LEFT JOIN article_labels al ON a.article_id=al.article_id
GROUP BY yr, al.label ORDER BY yr');

// category_dist: 카테고리별 성공/실패 상위 8개
const [catDist] = await db.execute(
`SELECT a.category, al.label, COUNT(*) cnt
FROM articles a LEFT JOIN article_labels al ON a.article_id=al.article_id
GROUP BY a.category, al.label ORDER BY cnt DESC LIMIT 50`);
};

```

4.5 기사 비교 GPT 스코어링 — compareController.js

★ 핵심 기술 — 6차원 GPT 채점 시장타이밍 / 실행력 / 고객이해도 / 경쟁대응력 / 자원충분성 / 트렌드부합도 — 0~100점.
NAVER 트렌드 CAT_MAP(17항목)으로 카테고리별 트렌드 컨텍스트 주입.

```

// compareController.js

const CAT_MAP = { // 기사 카테고리 → NAVER 트렌드 카테고리 매핑
'마케팅/브랜드': 'marketing', 'AI/디지털': 'IT',
'재무/투자': 'finance', ... // 17개 매핑
};

exports.compareArticles = async (req, res) => {
const { article1, article2 } = req.body;
const trendCtx = buildTrendContext(article1.category, article2.category);

// GPT 프롬프트 (JS 템플릿 리터럴)
// '두 전략 사례를 다음 6개 차원에서 각각 0~100으로 채점하세요.
// 차원: 시장타이밍, 실행력, 고객이해도, 경쟁대응력, 자원충분성, 트렌드부합도
// 트렌드 컨텍스트: ${trendCtx}
// 사례A: ${article1.title} 사례B: ${article2.title}'

// GPT JSON 반환: { scores: { A: {...}, B: {...}},
// analysis, key_differences, trend_insight, recommendation }
};

```

4.6 API 라우트 정의

```

// backend/src/routes/diagnoseRoutes.js

POST /api/diagnose/prompt-helper → verifyToken, promptHelper
POST /api/diagnose → verifyToken, diagnose
POST /api/diagnose/global → verifyToken, diagnoseGlobal
GET /api/diagnose/:id → verifyToken, getDiagnoseById
DELETE /api/diagnose/:id → verifyToken, deleteDiagnose

// backend/src/routes/authRoutes.js

POST /api/auth/login
POST /api/auth/register
GET /api/auth/kakao + /kakao/callback
GET /api/auth/naver + /naver/callback
GET /api/auth/google + /google/callback
POST /api/auth/refresh
POST /api/auth/logout (verifyToken 필수)

```

// 기타 라우트

GET /api/articles/stats → 기사 통계 (InsightDashboard)

GET /api/articles → 기사 목록 (페이지네이션 + 필터)

GET /api/articles/:id → 기사 단건

POST /api/compare → 기사 비교 GPT (CompareView)

GET /api/semantic-map/points → UMAP 전체 포인트 (SemanticMap)

5장. 프론트엔드 — frontend/ (React 18 + TypeScript)

React 18 + TypeScript + Tailwind CSS. D3.js Canvas+SVG 하이브리드 시맨틱맵 포함.

항목	내용
라우팅	History API pushState (SPA, 브라우저 뒤로가기 지원)
인증	apiFetch() — JWT Bearer 자동 첨부 + 401 시 Refresh Token 자동 재발급
상태 관리	React useState/useEffect (Redux 미사용, 가볍게 유지)
시맨틱맵	D3.js v7 Canvas(점 15,000+) + SVG(hull/레이블) 하이브리드
차트	Recharts (InsightDashboard 4개 차트, CompareView 6차원)
PDF 저장	jsPDF + html-to-image (DiagnosisResult, CompareView)

5.1 API 유틸리티 — api.ts (JWT 자동 갱신)

★ 핵심 기술 — JWT 자동 갱신 apiFetch()가 401 응답 시 tryRefresh()로 Refresh Token을 사용해 새 Access Token 발급, 원래 요청을 자동 재시도. 재발급 실패 시 localStorage 클리어 + 로그인 페이지 리다이렉트.

```
// frontend/src/app/utils/api.ts
export const BASE_URL = import.meta.env.PROD ? '' : 'http://localhost:3001';
// PROD 환경에서는 상대 URL '' → nginx 리버스 프록시 사용

async function tryRefresh(): Promise {
  const refresh = localStorage.getItem('refresh_token');
  if (!refresh) return false;
  const res = await fetch(`${BASE_URL}/api/auth/refresh`, {
    method: 'POST',
    body: JSON.stringify({ refresh_token: refresh }),
  });
  if (!res.ok) return false;
  const data = await res.json();
  localStorage.setItem('token', data.token);
  if (data.refresh_token) // Refresh Token Rotation 지원
    localStorage.setItem('refresh_token', data.refresh_token);
  return true;
}

export async function apiFetch(path: string, options = {}) {
  const token = localStorage.getItem('token');
  const res = await fetch(`${BASE_URL}${path}`, {
    ...options,
    headers: { Authorization: `Bearer ${token}`, ...options.headers },
  });
  if (res.status === 401) {
    const ok = await tryRefresh();
    if (ok) return apiFetch(path, options); // 원래 요청 재시도
    clearAuth(); window.location.href = '/'; // 재발급 실패 → 로그인아웃
  }
}
```



```
return res;
}
```

5.2 App.tsx — SPA 라우팅 + History API

★ 핵심 기술 — History API 기반 SPA 라우팅 window.history.pushState + popstate 이벤트로 브라우저 뒤로/앞으로 가기 지원. VIEW_PATHS 매핑(14개)으로 URL ↔ ViewType 변환. 소셜 콜백 시 popstate가 ?token= 파라미터를 지우지 않도록 URL 파라미터 체크 추가.

```
// frontend/src/app/App.tsx (핵심 라우팅 로직)
const VIEW_PATHS: Record = {
  dashboard: '/', risk: '/risk', analysis: '/analysis',
  'semantic-map': '/semantic-map', history: '/history',
  help: '/help', profile: '/profile', ... // 14개
};

function navigateTo(view: ViewType) {
  const path = VIEW_PATHS[view];
  window.history.pushState({ view, depth: depth+1 }, "", path);
  setCurrentView(view);
}

// 브라우저 뒤로/앞으로 버튼
useEffect(() => {
  const handlePop = (e: PopStateEvent) => {
    // 소셜 로그인 콜백 중이면 스킵 (token= 파라미터 보호)
    if (window.location.search.includes('token=') ||
        window.location.search.includes('error=')) return;
    const view = e.state?.view ?? 'dashboard';
    setCurrentView(view);
  };
  window.addEventListener('popstate', handlePop);
  return () => window.removeEventListener('popstate', handlePop);
}, []);
```

5.3 주요 컴포넌트 구조

컴포넌트	역할
DiagnosisInterview	전략 입력 폼 + AI 작성 도우미 (promptHelper API). 4단계 입력 → 진단 실행
DiagnosisResult	AI 리포트 컨설팅 문서 UI (01/02/03 섹션) + PDF 저장 (jsPDF+html-to-image)
SemanticMap	D3.js Canvas(점)+SVG(hull/레이블) 하이브리드. 15,000+점 고성능 렌더링
InsightDashboard	기사 통계 4개 차트 (Recharts: 파이/바/라인/복합). 기사 선택 → 비교
CompareView	6차원 GPT 스코어링 + 바/레이더 차트 + PDF 저장
AIChatbot	ChatGPT 스타일 전체화면 채팅. 세션 관리 + 대화 연속성 (DB 저장)
SearchHistory	진단이력 목록. 클릭 시 결과 복원 (report_json 전체 저장)
TopNavigation	탭 네비게이션 + JWT 상태 + 다크모드 + 언어 전환 + 알림

★ SemanticMap 핵심 기술 — D3.js Canvas+SVG 하이브리드 Canvas: 15,000+ 점을 GPU 가속으로 고성능 렌더링 (success=에메랄드, failure=빨강 glow). SVG: convex hull 경계 + 클러스터 레이블 (실패율 기반 색상: 빨강 $\geq 25\%$ /주황 $\geq 12\%$ /인디고). d3.quadtree()로 마우스 호버 최근접 점 탐색. d3.zoom()으로 pan/zoom.

6장. DB 스키마 핵심 테이블 (MySQL)

테이블	주요 컬럼
users	user_id, email, password_hash (NULL가능), user_type, subscription_type, refresh_token, notif_* 3개
articles	article_id, article_no(UNIQUE MD5), title, content, summary, url, company_name, industry, strategy_type, published_at, source, category
article_labels	label_id, article_id(FK), label(success/failure/neutral), label_method(auto/manual), confidence
article_vectors	vector_id, article_id(FK,UNIQUE), embedding_vector(JSON), umap_x, umap_y, cluster_id(FK)
clusters	cluster_id, cluster_name, representative_industry, top_keywords, article_count, center_x, center_y
diagnosis_requests	diagnosis_id, user_id(FK), input_text, industry, status(pending/processing/completed/failed)
analysis_results	result_id, diagnosis_id(FK), risk_score, analysis_mode, keywords, improvement, query_umap_x, query_umap_y, query_cluster_id, report_json(MEDIUMTEXT)
similar_article_matches	match_id, result_id(FK), article_id(FK), similarity_score, rank
strategies	strategy_id, user_id(FK), name, content, keywords(JSON), metrics_*
chat_sessions	session_id, user_id(FK), title(200자), created_at, updated_at
chat_messages	message_id, session_id(FK), role(user/assistant), content, created_at
feedbacks	feedback_id, diagnosis_id(FK), user_id(FK), rating, comment

analysis_results.report_json (MEDIUMTEXT): GPT 리포트 전체를 JSON 직렬화하여 저장. 진단이력 복원 시 GPT 재호출 없이 전체 리포트 즉시 표시 가능.

7장. 배포 구성 — NCP + Docker

NCP(Naver Cloud Platform) G3 서버 + Docker Compose 3컨테이너 구성.

항목	내용
서버 IP	211.188.50.81 (현재 정지 상태, 크레딧 절약)
frontend	node:20-slim 빌드 → nginx:alpine 서빙. 포트 80:80 노출
backend	expose 3001 (내부), nginx /api/* 리버스 프록시
ai_server	expose 8000 (내부), healthcheck 설정, 시작 ~60초 소요
네트워크	dongajul_net 내부 브릿지. backend↔ai_server HTTP 통신

```
# docker-compose.yml 핵심 구조

services:
  frontend:
    build: ./frontend
    ports: ['80:80'] # 외부 노출은 frontend만
    networks: [dongajul_net]

  backend:
    build: ./backend
    expose: ['3001'] # 내부 전용
    environment:
      - AI_SERVER_URL=http://ai_server:8000
    networks: [dongajul_net]

  ai_server:
    build: ./ai_server
    expose: ['8000'] # 내부 전용
    healthcheck:
      test: curl -f http://localhost:8000/health
    networks: [dongajul_net]

# nginx.conf 핵심
location /api/ {
  proxy_pass http://backend:3001; # /api/* → backend
}

location / {
  try_files $uri $uri/ /index.html; # SPA 라우팅
}
```

```
# NCP 서버 재시작 → 서비스 시작
ssh root@211.188.50.81
cd /root/dongajul
docker compose start

# ai_server 로드 약 60초 후 http://211.188.50.81 접속

# 코드 업데이트 배포
git pull origin develop
docker compose build --no-cache
```

`docker compose up -d`

동아줄 — AI 전략 리스크 진단 서비스 · 전체 코드 해설집

데이터: DBR 11,273건 / HBR 2,062건 / HBS 2,116건 / NAVER 55,465건

모델: jhgan/ko-sroberta-multitask (768차원) + FAISS IndexFlatIP + MLP (ROC-AUC 0.9771)